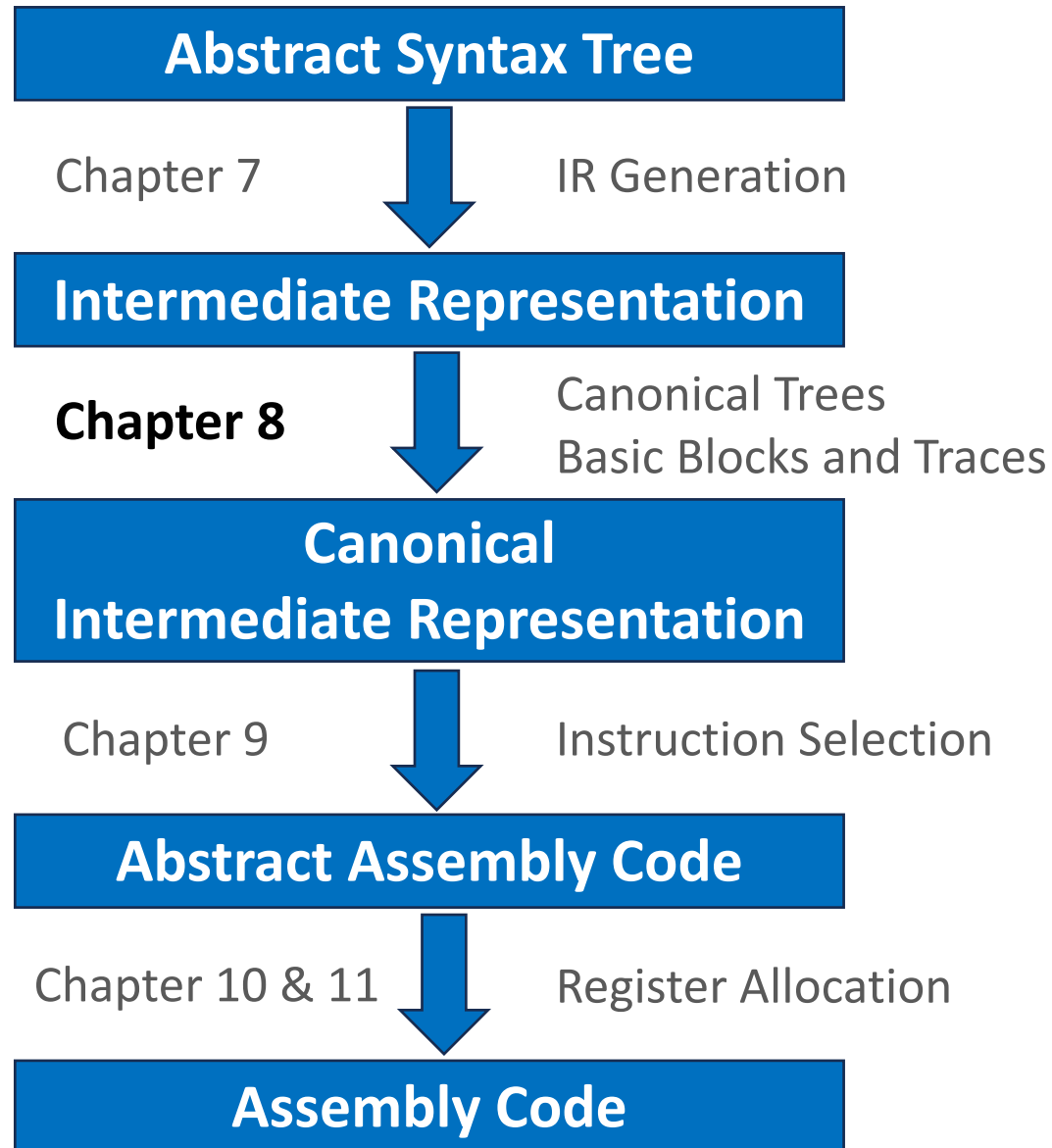


Basic Blocks and Traces

Chapter 8

Where We Are

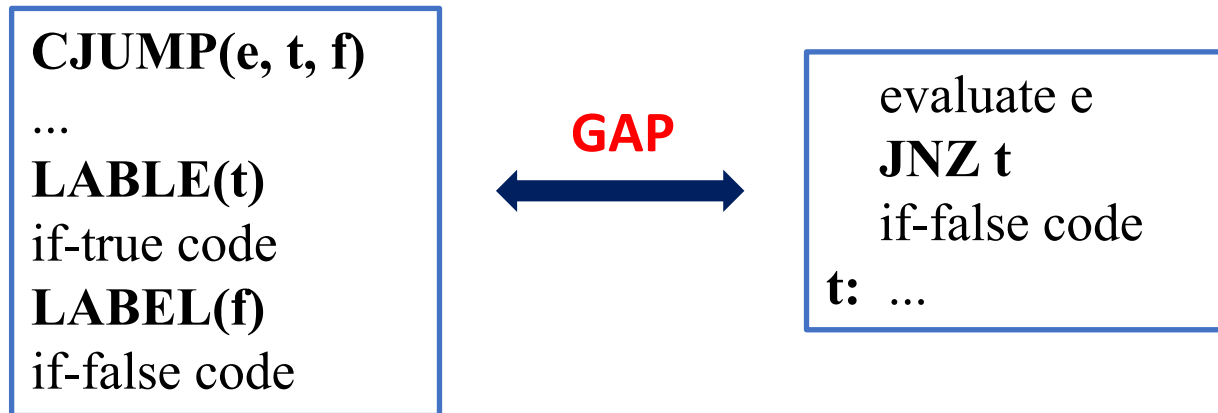


Motivation

- IR must be translated into assembly or machine code.
- The operators of the *Tree* language are chosen carefully to match the capabilities of most machines.
- However,
 - Some aspects of the *Tree* language do not correspond exactly with **machine languages**
 - Some aspects of the *Tree* language interfere with **compile-time optimization analyses**.

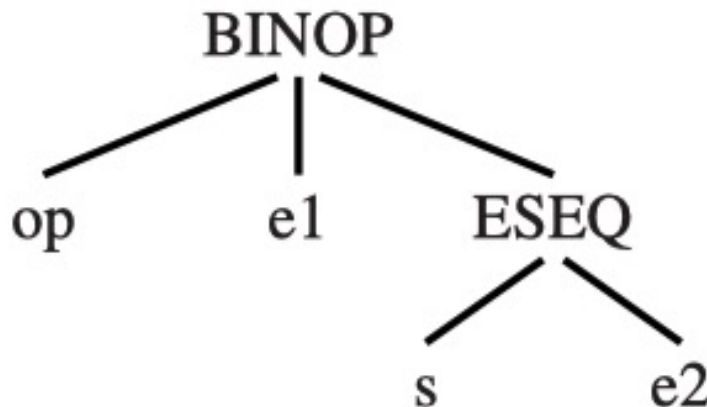
Mismatches: IR Trees vs. Machine Code

1. The **CJUMP** instruction can jump to either of two labels.
 - Real machines' conditional jump instructions fall through to the next instruction if the condition is false.



Mismatches: Trees vs. Machine Code

2. **ESEQ** nodes within expressions are inconvenient.
 - Statements have side effects.
 - Different orders of evaluating subtrees yield different results.
 - But it is useful to be able to evaluate the subexpressions of an expression in any order.
3. **CALL** nodes within expressions cause the same problem.



E.g., `e1: a + 5`
`s: a = 5`

Mismatches: Trees vs. Machine Code

4. **CALL** nodes within the argument-expressions of other **CALL** nodes will cause problems when trying to **put arguments into a fixed set of formal-parameter registers**.

- e.g., **CALL**(f, [e1, **CALL**(g, [e2, ...])])
- Assume the result of **e1** is stored in **r1**, the result of **e2** will overwrite **r1**

Mismatches: Trees vs. Machine Code

- The **CJUMP** instruction can jump to either of two labels.
 - Real machines' conditional jump instructions fall through to the next instruction if the condition is false.
- **ESEQ** nodes within expressions are inconvenient.
 - Different orders of evaluating subtrees yield different results.
 - But it is useful to be able to evaluate the subexpressions of an expression in any order.
- **CALL** nodes within expressions cause the same problem.
- **CALL** nodes within the argument-expressions of other **CALL** nodes will cause problems when trying to **put arguments into a fixed set of formal-parameter registers**.
 - e.g., **CALL**(f, [e1, **CALL**(g, [e2, ...])])

Solution

- How to eliminate these mismatches?
- We transform the tree in three stages:
 1. A tree is rewritten into a list of *canonical trees* without SEQ or ESEQ nodes
 - Eliminating mismatches related to ESEQ and CALL
 2. This list is grouped into a set of *basic blocks*, which contain no internal jumps or labels
 3. The basic blocks are ordered into a set of *traces* in which every CJUMP is immediately followed by its false label.
 - Eliminating mismatches related to CJUMP

Outline

- **Canonical Trees**
- Taming Conditional Branches
 - Basic Blocks
 - Traces

Canonical Trees

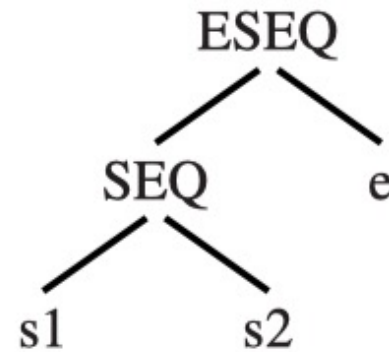
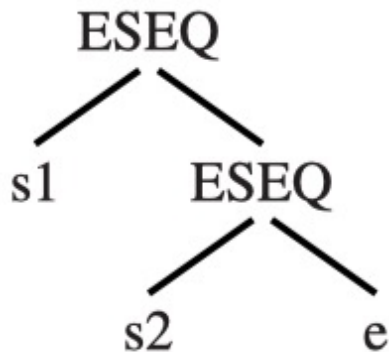
- **Stage one:** A tree is rewritten into a list of *canonical trees*
- **Canonical Trees** are defined as having these properties:
 1. No **SEQ** or **ESEQ**
 2. The parent of each **CALL** is either **EXP(...)** or **MOVE(TEMP t, ...)**
- *Property 1:*
 - Each canonical tree contains at most one statement node, i.e., the root node. Other nodes are all expression nodes.
- *Property 2:*
 - The children of a **CALL** node cannot be **CALL** nodes
- *Property 1 and property 2:*
 - The parent of a **CALL** node must be the root node of a canonical tree.
 - There can be at most one **CALL** node in a canonical tree, because **EXP(...)** and **MOVE(TEMP t, ...)** can only contains one **CALL**.

Canonical Trees

- **Stage one:** A tree is rewritten into a list of *canonical trees*
- To perform stage-one transformation, we need to:
 - **eliminate ESEQs**
 - move CALLs to top level
 - eliminate SEQs

Transformations on ESEQ

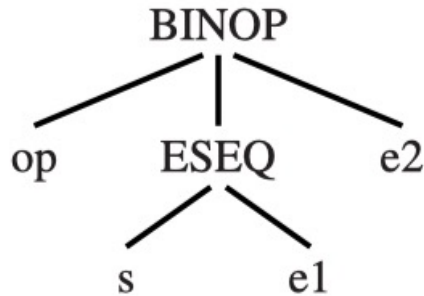
- How can the **ESEQ** nodes be eliminated?
- **ESEQ**(s , e)
- The idea is to lift the **ESEQ** nodes higher and higher in the tree, until they can become **SEQ** nodes.



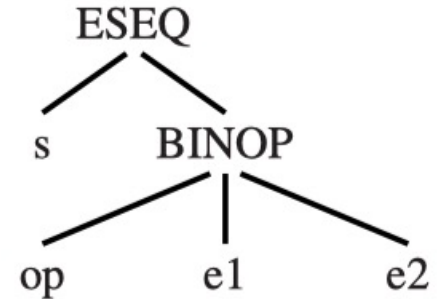
$\text{ESEQ}(s_1, \text{ESEQ}(s_2, e))$

$\text{ESEQ}(\text{SEQ}(s_1, s_2), e)$

Transformations on ESEQ



$\text{BINOP}(op, \text{ESEQ}(s, e_1), e_2)$



$\text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$

$\text{MEM}(\text{ESEQ}(s, e_1))$

$\text{JUMP}(\text{ESEQ}(s, e_1))$

$\text{CJUMP}(op, \text{ESEQ}(s, e_1), e_2, l_1, l_2)$



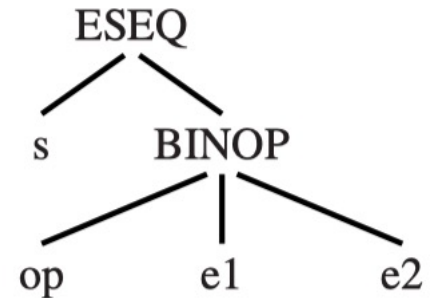
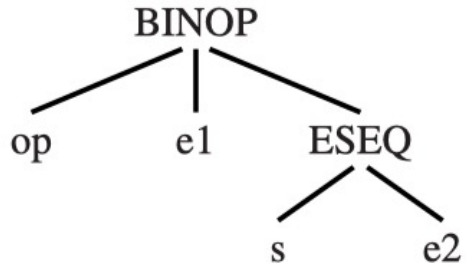
$\text{ESEQ}(s, \text{MEM}(e_1))$

$\text{SEQ}(s, \text{JUMP}(e_1))$

$\text{SEQ}(s, \text{CJUMP}(op, e_1, e_2, l_1, l_2))$

- Given $\text{ESEQ}(s, e)$:
 - extract s
 - rewrite its parent as ESEQ or SEQ with s

Transformations on ESEQ



$\text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$

$\text{BINOP}(op, e_1, \text{ESEQ}(s, e_2))$

$\text{CJUMP}(op, e_1, \text{ESEQ}(s, e_2), l_1, l_2)$

?

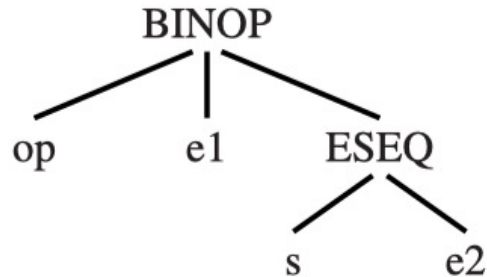
Suppose:

$s = \text{MOVE}(\text{MEM}(x), y)$

$e1 = \text{MEM}(x)$

- To preserve the order of evaluation, we must pull **e1** out of the BINOP with **s**
- How?

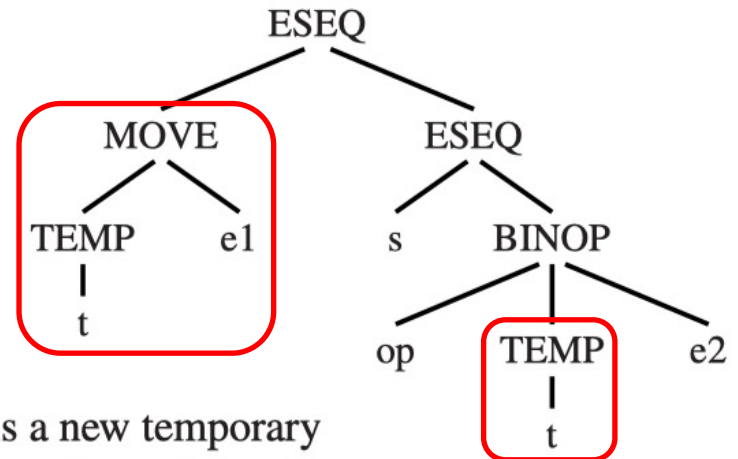
Transformations on ESEQ



$\text{BINOP}(op, e1, \text{ESEQ}(s, e2))$



Intermediate result



t is a new temporary

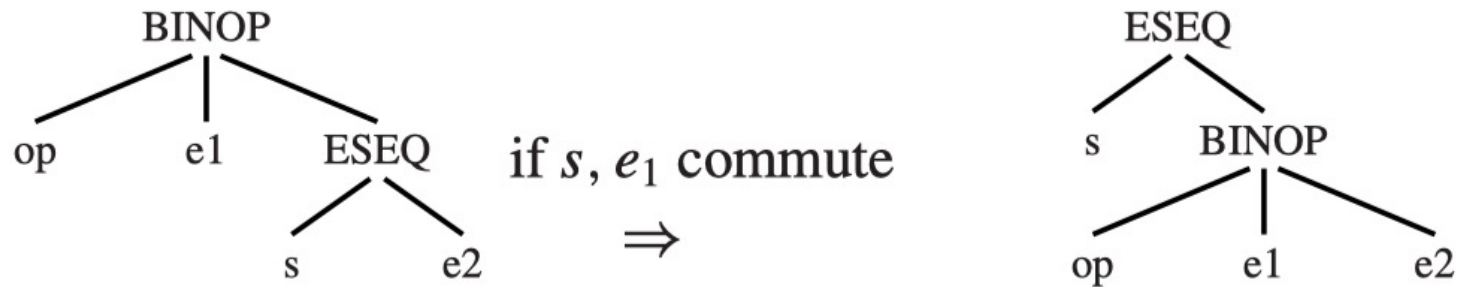
$\text{ESEQ}(\text{MOVE}(\text{TEMP } t, e1),$
 $\text{ESEQ}(s, \text{BINOP}(op, \text{TEMP } t, e2)))$

$\text{CJUMP}(op, e1, \text{ESEQ}(s, e2), l_1, l_2)$



$\text{SEQ}(\text{MOVE}(\text{TEMP } t, e1),$
 $\text{SEQ}(s, \text{CJUMP}(op, \text{TEMP } t, e2, l_1, l_2)))$

Transformations on ESEQ



$$\text{if } s, e_1 \text{ commute} \\ \text{BINOP}(op, e_1, \text{ESEQ}(s, e_2)) = \text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$$

$$\text{CJUMP}(op, e_1, \text{ESEQ}(s, e_2), l_1, l_2) = \text{SEQ}(s, \text{CJUMP}(op, e_1, e_2, l_1, l_2))$$

- If the temporaries and memory locations assigned by s are not referenced by $e1$ (and s and $e1$ don't both perform external I/O)

Transformations on ESEQ

- How to know whether a statement s commutes with an expression e ?
- We cannot always know this at compile time, e.g.,:
 - $s = \text{MOVE}(\text{MEM}(x), y)$
 - $e = \text{MEM}(z)$
 - Whether $x == z$?
- We **conservatively** approximate whether statements commute
 - $\text{commute}(s, e) = \text{True}$ if s and e definitely do commute
 - $\text{commute}(s, e) = \text{False}$ otherwise

Transformations on ESEQ

- The **commute** function estimates (very naively) whether a statement commutes with an expression:
 - A **constant** commutes with any statement
 - An **empty statement** commutes with any expression.
 - **EXP(CONST X)**
 - Anything else is assumed not to commute

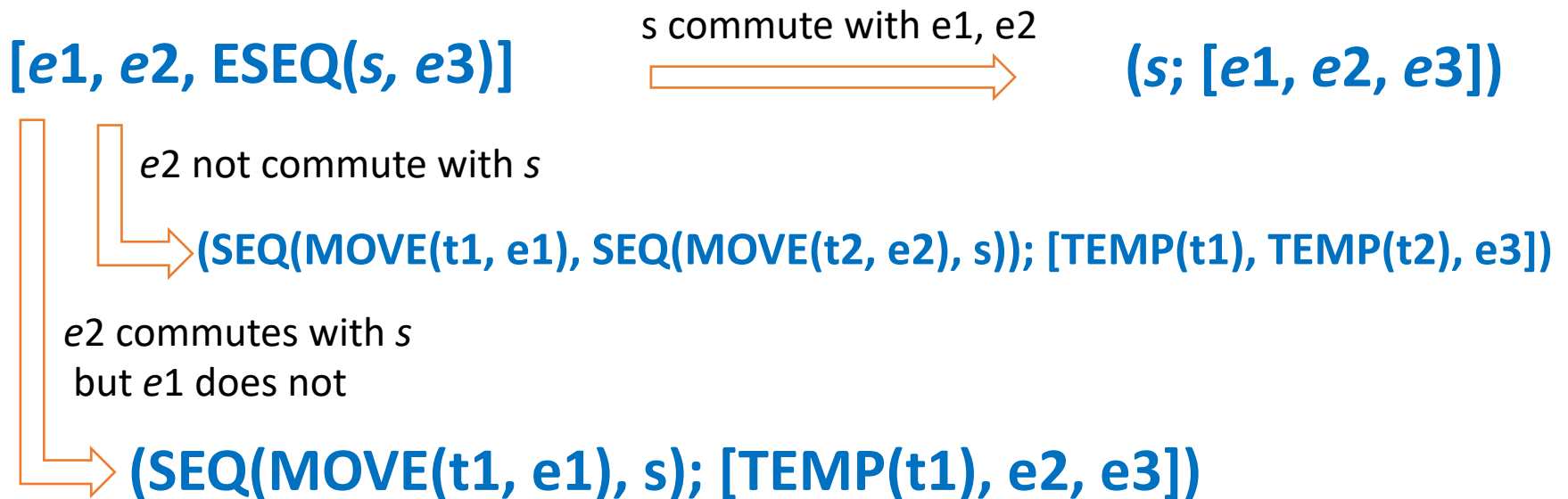
```
static bool isNop(T_stm x) {  
    return x->kind == T_EXP && x->u.EXP->kind == T_CONST;  
}  
  
static bool commute(T_stm x, T_exp y){  
    return isNop(x) || y->kind == T_NAME || y->kind == T_CONST;  
}
```

General Rewriting Rules

- For each kind of Tree statement or expression, similar rules can be made to pull **ESEQs** out of the statement or expression.
- Given a Tiger program (which is a **T_stm**), we can recursively perform this transformation to lift all **ESEQ** higher and higher in the tree, until they can become **SEQ** nodes.
- How to rewrite each kind of statement or expression?

General Rewriting Rules

- In general, for each kind of Tree stmt/exp, we can identify its **subexpressions**.
 - e.g., **[e1, e2, ESEQ(s, e3)]**
- **Rewrite:** Extracts the “statement” in its subexpressions and update the subexpressions of the stmt/exp.



- Tiger compiler defines a **reorder** function to implement this procedure.

General Rewriting Rules

Algorithm

- Step 1: Make a “subexpression-extraction” method for each kind of Tree statement or expression
 - Extract the “statement” in the subexpressions and convert each subexpression into its ESEQ-clean version.
- Step 2: Make a “subexpression-insertion” method:
 - Build a new version of the statement or expression with the ESEQ-clean version of each subexpression.
- The extracted statement and the new version of the statement or expression will be concatenated with SEQ or ESEQ nodes.
- *For more details, refer to the backup slides and the textbook*

Canonical Trees

- **Stage one:** A tree is rewritten into a list of *canonical trees*
- To perform stage-one transformation, we need to:
 - eliminate ESEQs
 - **move CALLs to top level**
 - eliminate SEQs

Move CALLs to Top Level

- The Tree language permits CALL nodes to be used as subexpressions
- However, each function returns its result in the same dedicated return-value register **TEMP(RV)**

BINOP(PLUS, CALL(...), CALL(...))

- The second call will **overwrite** the **RV** register before the PLUS can be execute
- How to solve this problem?

Move CALLS to Top Level

- **Idea:** assign each return value immediately into a fresh temporary register:

CALL(f, args) ->

ESEQ(MOVE(TEMP t, CALL(f, args)), TEMP t)

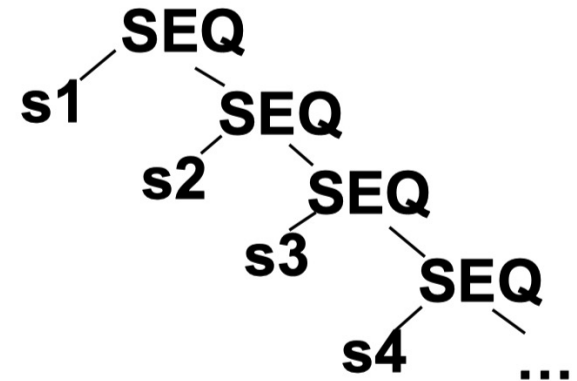
Canonical Trees

- **Stage one:** A tree is rewritten into a list of *canonical trees*
- To perform stage-one transformation, we need to:
 - eliminate ESEQs
 - move CALLs to top level
 - **eliminate SEQs**

A Linear List of Statements

- Once an entire function body s_0 is processed with the rewriting rules, the result is a tree s_0' where all the **SEQ** nodes are near the top

- $\text{SEQ}(\text{SEQ}(\text{SEQ}(\dots, s_x), s_y), s_z)$



- We repeatedly applies the rule:

$$\text{SEQ}(\text{SEQ}(a, b), c) = \text{SEQ}(a, \text{seq}(b, c))$$

- s_0' is linearized into a statement of the form:
 - $\text{SEQ}(s_1, \text{SEQ}(s_2, \dots, \text{SEQ}(s_{n-1}, s_n) \dots))$
 - provides no structuring information*
- We can just consider this to be a simple list of statements:
 - $s_1, s_2, \dots, s_{n-1}, s_n$
 - None of the s_i contain **SEQ** or **ESEQ** nodes

Outline

- Canonical Trees
- **Taming Conditional Branches**
 - **Basic Blocks**
 - Traces

Solution

- How to eliminate these mismatches?
- We transform the tree in three stages:
 - ~~1. A tree is rewritten into a list of *canonical trees* without SEQ or ESEQ nodes~~
 - ~~• Eliminating mismatches related to ESEQ and CALL~~
 2. This list is grouped into a set of *basic blocks*, which contain no internal *jumps* or *labels*
 3. The basic blocks are ordered into a set of *traces* in which every *CJUMP* is immediately followed by its false label.
- The last two stages make *CJUMP*(cond, l_t , l_f) is immediately followed by *LABEL*(l_f).
 - Each such *CJUMP* can be directly implemented on a real machine as a conditional branch to l_t

Solution

- How to eliminate these mismatches?
- We transform the tree in three stages:
 - ~~1. A tree is rewritten into a list of *canonical trees* without SEQ or ESEQ nodes~~
 - ~~• Eliminating mismatches related to ESEQ and CALL~~
 - 2. This list is grouped into a set of *basic blocks*, which contain no internal *jumps* or *labels***
 - 3. The basic blocks are ordered into a set of *traces* in which every *CJUMP* is immediately followed by its false label.**
- The last two stages make **CJUMP(cond, l_t , l_f)** is immediately followed by **LABEL(l_f)**.
 - Each such CJUMP can be directly implemented on a real machine as a conditional branch to l_t

Basic Blocks - Motivation

- In determining where the jumps go in a program, we are analyzing the program's **control flow**.
- **Control flow** is the sequencing of instructions in a program
 - ignoring the data values in registers and memory
 - ignoring the arithmetic calculations
- We cannot know whether the conditional jumps will go to their true or false labels; so we simply say that such jumps can go either way.
- In analyzing the control flow of a program, any instruction that is not a jump has an entirely uninteresting behavior.
- Lump together any sequence of non-branch instructions into a basic block and analyze the control flow between basic blocks.

Basic Blocks - Definition

- A **basic block** is a sequence of statements that is always entered at the beginning and exited at the end, that is:
 - The first statement is a LABEL
 - The last statement is a JUMP or CJUMP
 - There are no other LABELs, JUMPs, or CJUMPs

LABEL XX

... # no LABELs, JUMPs, CJUMPs

JUMP / CJUMP

Basic Blocks - Algorithm

- The **algorithm** for dividing a long sequence of statements into basic blocks is as follows:
 - Scan the sequence from beginning to end;
 - Whenever a LABEL is found, a new block is started (the previous block is ended);
 - Whenever a JUMP or CJUMP is found, a block is ended (and the next block is started).
 - If this leaves any block not ending with a JUMP or CJUMP, then a JUMP to the next block's label is appended to the block.
 - If any block has been left without a LABEL at the beginning, a new label is invented and stuck here.
- Why do we need to append JUMP or invent LABEL?

Basic Blocks – Example

```
(1) x := input
(2) y := x - 1
(3) z := x * y
(4) if z < x goto (7)
(5) p := x / y
(6) q := p + y
(7) a := q
(8) b := x + a
(9) c := a - b
(10) if p == q goto (12)
(11) goto (3)
(12) return
```

Three address code

Algorithm:

- Scanning the sequence from beginning to end;
- Whenever a LABEL is found, a new block is started (the previous block is ended);
- Whenever a JUMP or CJUMP is found, a block is ended (and the next block is started);
- Add LABELs and JUMPs.

Basic Blocks – Example

```
(1) x := input
(2) y := x - 1
(3) z := x * y
(4) if z < x goto (7)
(5) p := x / y
(6) q := p + y
(7) a := q
(8) b := x + a
(9) c := a - b
(10) if p == q goto (12)
(11) goto (3)
(12) return
```

Three address code

Algorithm:

- Scanning the sequence from beginning to end;
- Whenever a LABEL is found, a new block is started (the the previous block is ended);
 - start from (3), (7), (12)

Basic Blocks – Example

(1) $x := \text{input}$

(2) $y := x - 1$

(3) $z := x * y$

(4) if $z < x$ goto (7)

(5) $p := x / y$

(6) $q := p + y$

(7) $a := q$

(8) $b := x + a$

(9) $c := a - b$

(10) if $p == q$ goto (12)

(11) goto (3)

(12) return

Algorithm:

- Scanning the sequence from beginning to end;
- Whenever a LABEL is found, a new block is started (the the previous block is ended);
 - start from (3), (7), (12)
- Whenever a JUMP or CJUMP is found, a block is ended (and the next block is started).
 - end at (4), (10), (11) (and start from (5), (11), (12))

Basic Blocks – Example

(1) $x := \text{input}$

(2) $y := x - 1$

(3) $z := x * y$

(4) if $z < x$ goto (7)

(5) $p := x / y$

(6) $q := p + y$

(7) $a := q$

(8) $b := x + a$

(9) $c := a - b$

(10) if $p == q$ goto (12)

(11) goto (3)

(12) return

Algorithm:

- Scanning the sequence from beginning to end;
- Whenever a LABEL is found, a new block is started (the the previous block is ended);
 - start from (3), (7), (12)
- Whenever a JUMP or CJUMP is found, a block is ended (and the next block is started).
 - end at (4), (10), (11) (and start from (5), (11), (12))
- Add LABELs and JUMPs
 - we ignore in this example
 - **should be done for Tree lang**

Basic Blocks – Example

```
(1) x := input
(2) y := x - 1
(3) z := x * y
(4) if z < x goto (7)
(5) p := x / y
(6) q := p + y
(7) a := q
(8) b := x + a
(9) c := a - b
(10) if p == q goto (12)
(11) goto (3)
(12) return
```



```
(1) x := input
(2) y := x - 1
```

```
(3) z := x * y
(4) if z < x goto (7)
```

```
(5) p := x / y
(6) q := p + y
```

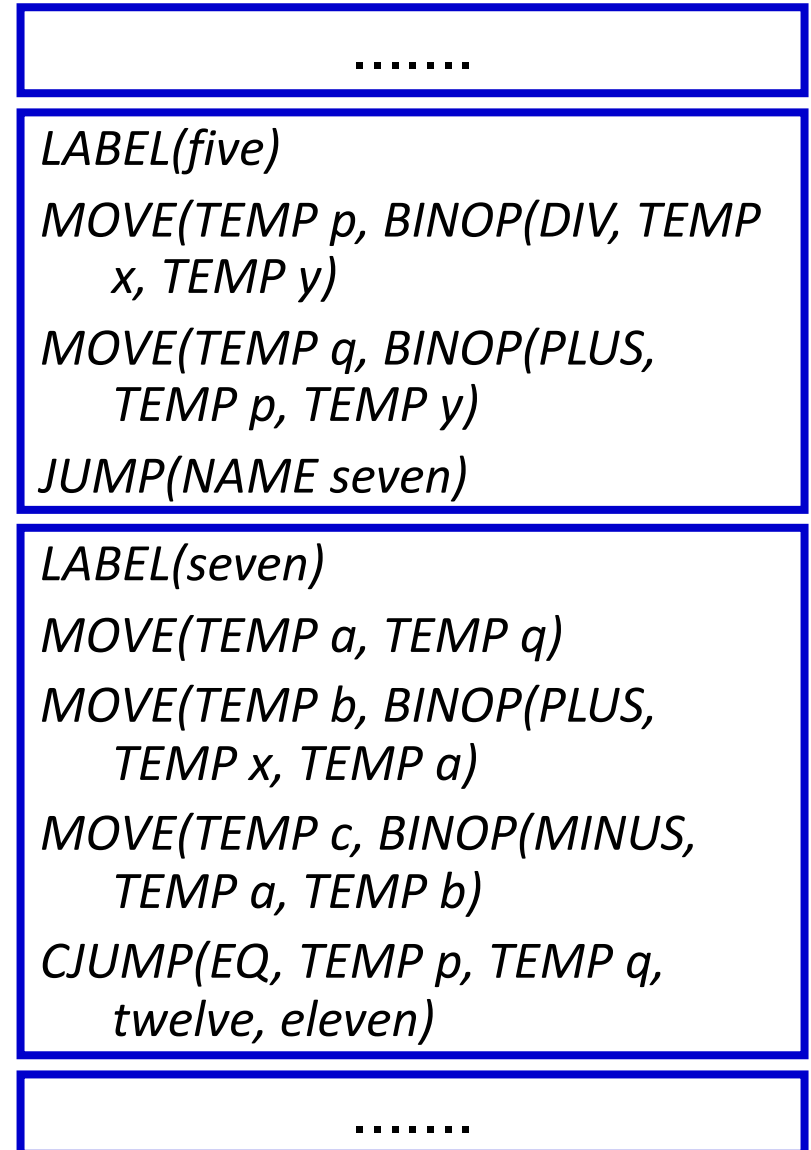
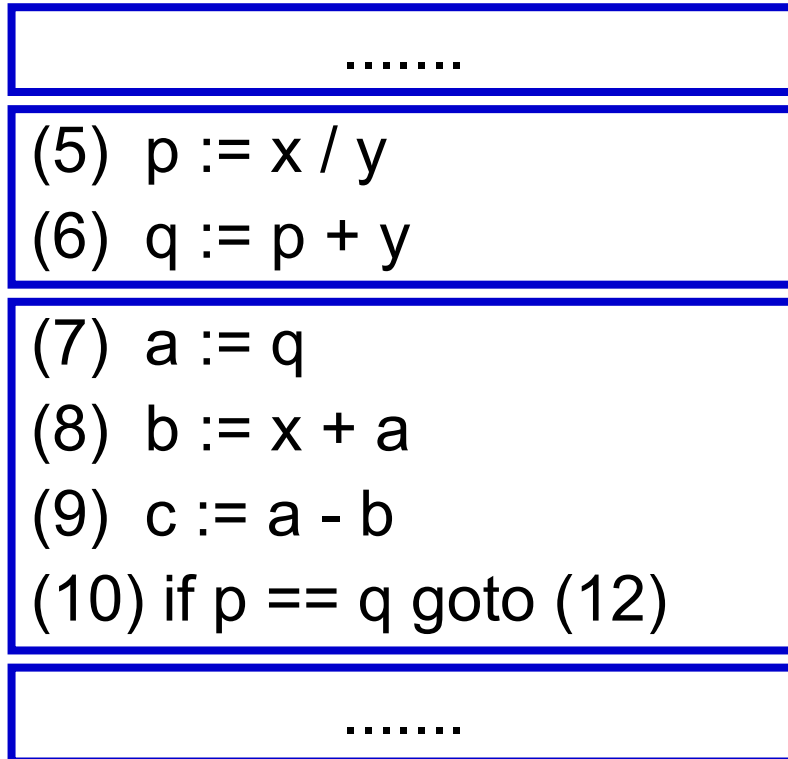
```
(7) a := q
(8) b := x + a
(9) c := a - b
(10) if p == q goto (12)
```

```
(11) goto (3)
```

```
(12) return
```

start from: (1), (3), (5), (7), (11), (12)

Basic Blocks - Example



Tree Program with LABELs and JUMPs added

Outline

- Canonical Trees
- **Taming Conditional Branches**
 - Basic Blocks
 - **Traces**

Solution

- How to eliminate these mismatches?
- We transform the tree in three stages:
 - ~~1. A tree is rewritten into a list of *canonical trees* without SEQ or ESEQ nodes~~
 - ~~• eliminating mismatches related to ESEQ and CALL~~
 - ~~2. This list is grouped into a set of *basic blocks*, which contain no internal jumps or labels~~
 - 3. The basic blocks are ordered into a set of *traces* in which every **CJUMP** is immediately followed by its false label.**
- The last two stages make $\text{CJUMP}(\text{cond}, l_t, l_f)$ immediately followed by $\text{LABEL}(l_f)$.
 - Each such CJUMP can be directly implemented on a real machine as a conditional branch to l_t

Traces - Motivation

The basic blocks can be arranged in any order, and the result of executing the program will be the same.

- Based on this, we can choose an ordering of the blocks satisfying the condition that each **CJUMP** is followed by its false label.

```
CJUMP(op, a, b, lt, lf)  
LABEL(lf)
```

- We can also arrange that many of the unconditional **JUMPs** are immediately followed by their target label.
 - Allow the deletion of these jumps, making the compiled program run a bit faster.

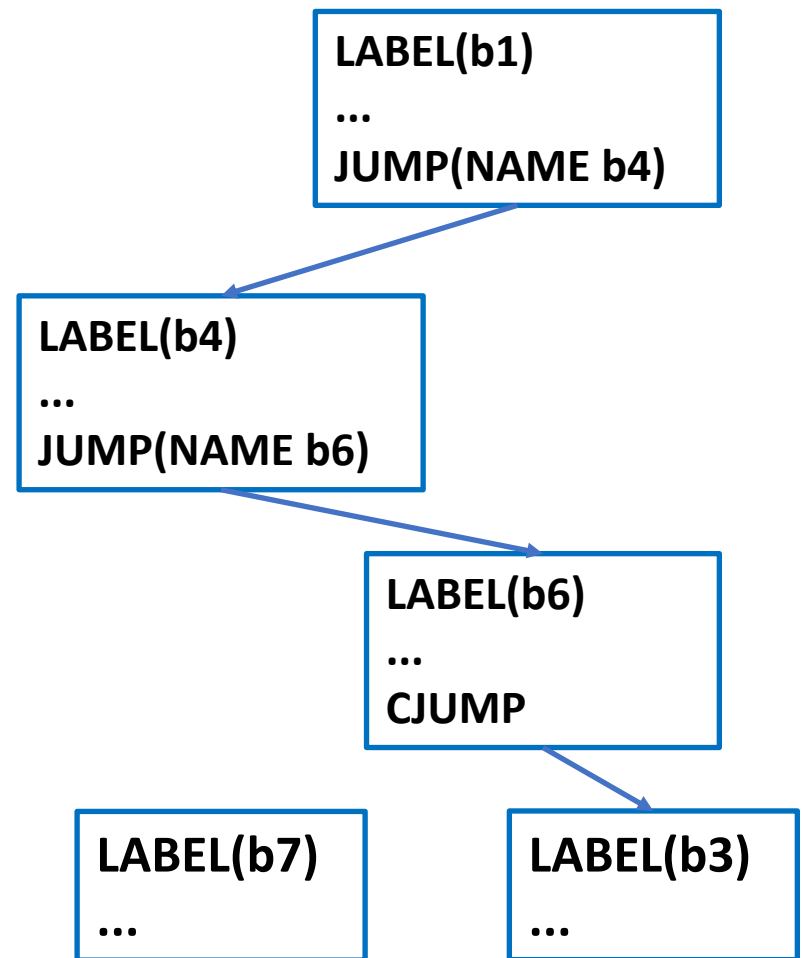
```
...  
JUMP(NAME next)  
LABEL(next)
```

Traces - Definition

- A **trace** is a sequence of statements that could be consecutively executed during the execution of the program and can include conditional branches.
- A program has many different, overlapping traces.
- For our purposes in arranging **CJUMPs** and false-labels, we want to make a set of traces that exactly covers the program:
 - Each block must be in exactly one trace.
 - Each trace is loop free.
- We would like to have as few traces as possible in our covering set.
 - To minimize the number of JUMPs from one trace to another

Traces - Algorithm

- How to find a covering set of traces?
- The idea is to start with some block - the beginning of a trace - and follow a possible execution path - the rest of the trace.
- Suppose block $b1$ ends with a JUMP to $b4$, and $b4$ has a JUMP to $b6$. Then we can make the trace $b1, b4, b6$.
- Suppose $b6$ ends with a conditional jump CJUMP(cond, $b7, b3$). We append $b3$ to our trace and continue with the rest of the trace after $b3$.
 - make CJUMP(cond, l_t, l_f) is immediately followed by LABEL(l_f)
- The block $b7$ will be in some other trace.



Traces - Algorithm

Put all the blocks of the program into a list Q .

while Q is not empty

 Start a new (empty) trace, call it T .

DFS

 Remove the head element b from Q .

while b is not marked

 Mark b ; append b to the end of the current trace T .

 Examine the successors of b (the blocks to which b branches);

if there is any unmarked successor c

$b \leftarrow c$

 (*All the successors of b are marked.*)

 End the current trace T .

ALGORITHM 8.3. Generation of traces.

- Start with some block and follow a chain of jumps, marking each block and appending it to the **current trace**.
- When coming to a block whose successors are all marked, the generation of **one trace is finished** -> Pick an unmarked block to start the **next trace**

Finishing Up

To simplify the implementation of later phases, the Tiger compiler flattens the ordered list of traces back into one long list of statements.

Perform some minor adjustments:

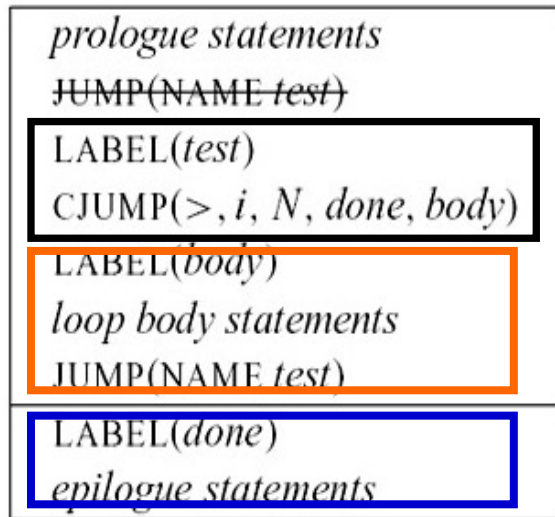
- For any **CJUMP** immediately followed by its false label:
 - Let alone (there will be many of these).
- For any **CJUMP** followed by its true label:
 - Switch the true and false labels and negate the condition.
- For any **CJUMP(*cond*, *a*, *b*, *lt*, *lf*)** followed by neither label:
 - invent a new false label *lf'*
 - rewrite the single **CJUMP** statement as three statements:

```
CJUMP(cond, a, b, lt, lf')  
LABEL lf'  
JUMP(NAME lf)
```

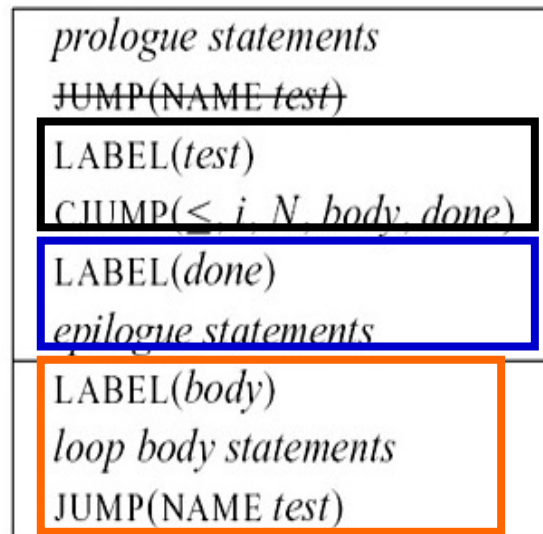
Optimal Traces

- Any frequently executed sequence of instructions (such as the body of a loop) should occupy its own trace.
 - This helps to minimize the number of unconditional jumps.
 - This helps with other kinds of optimizations.
 - register allocation
 - instruction scheduling
 - ...

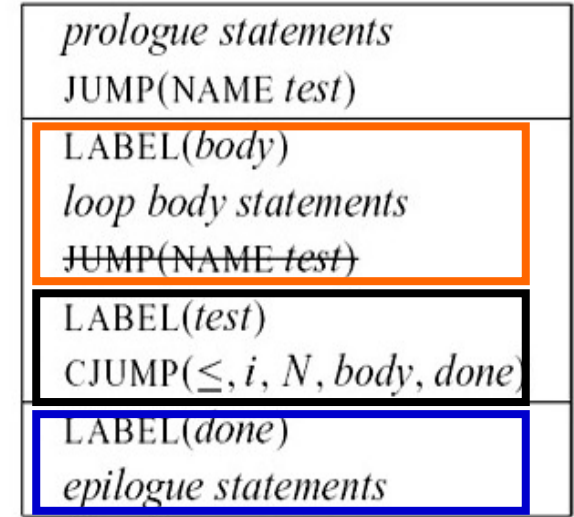
Optimal Traces



(a)



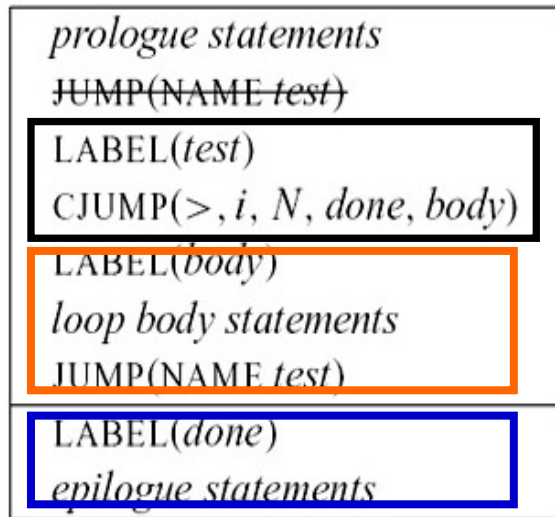
(b)



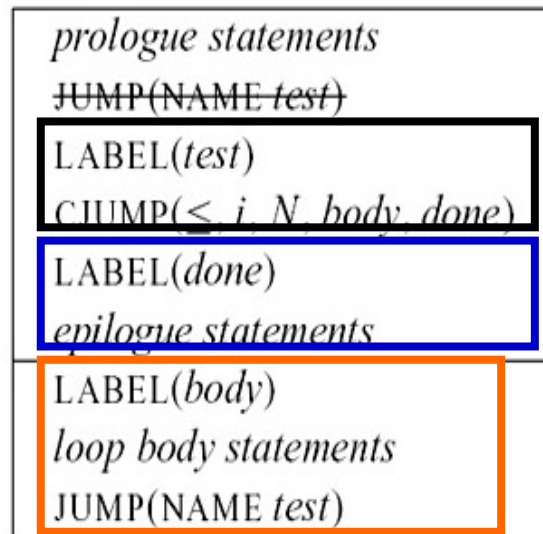
(c)

- Which is better?
 - Which one contains the least jumps?

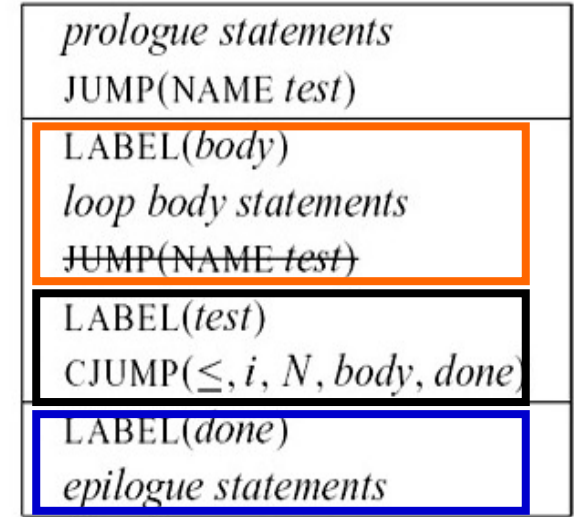
Optimal Traces



(a)



(b)



(c)

- (a) Each while loop contains a **CJUMP** and a **JUMP**
- (b) Use different traces, but each while loop still contains a **CJUMP** and a **JUMP**
- (c) 1st iteration: a **CJUMP** and a **JUMP**; Other iterations: only a **CJUMP**

Summary

- **Problem:** mismatches between IR trees and machine instructions
 - CJUMP v.s. machine conditional jump
 - The order of subtree evaluation in ESEQ and CALL matters.
 - CALL as arguments to another CALL
- **Approach:** Transform the tree in three stages
 1. A tree is rewritten into a list of *canonical trees* without SEQ or ESEQ nodes
 - Eliminating mismatches related to ESEQ and CALL
 2. This list is grouped into a set of *basic blocks*, which contain no internal jumps or labels
 3. The basic blocks are ordered into a set of *traces* in which every CJUMP is immediately followed by its false label.
 - Eliminating mismatches related to CJUMP

Homework

- **8.2, 8.6, 8.7**

Rules for Canonical Tree Construction

ESEQ(s1, ESEQ(s2, e))	⇒	ESEQ(SEQ(s1,s2), e))
BINOP(op, ESEQ(s, e1), e2)	⇒	ESEQ(s, BINOP(op, e1, e2))
MEM(ESEQ(s,e1))	⇒	ESEQ(s, MEM(e1))
JUMP(ESEQ(s, e1))	⇒	SEQ(s, JUMP(e1))
CJUMP(op, ESEQ(s, e1), e2, l1,l2)	⇒	SEQ(s, CJUMP(op, e1, e2, l1,l2))
BINOP(op, e1, ESEQ(s, e2))	⇒	ESEQ(MOVE(TEMP t, e1), ESEQ(s, BINOP(op, TEMP t, e2)))
CJUMP(op, e1, ESEQ(s, e2), l1, l2)	⇒	SEQ(MOVE(TEMP t, e1), SEQ(s, CJUMP(op, TEMP t, e2, l1,l2)))
MOVE(ESEQ(s, e1), e2)	⇒	SEQ(s, MOVE(e1, e2))
CALL(f , a)	⇒	ESEQ(MOVE(TEMP t, CALL(f , a)), TEMP t)

Canon Module

```
typedef struct C_stmListList_ *C_stmListList;
struct C_block { C_stmListList stmLists; Temp_label label;};
struct C_stmListList_ { T_stmList head; C_stmListList
tail;};

T_stmList C_linearize(T_stm stm);
struct C_block C_basicBlocks(T_stmList stmList);
T_stmList C_traceSchedule(struct C_block b);
```

The interface of the **Canon** module:

- *linearize*: removes the **ESEQs** and moves the **CALLs** to top level
- *basicBlocks*: groups statements into sequences of straight-line code
- *traceSchedule*: orders the blocks so that every **CJUMP** is followed by its false label

General Rewriting Rules

```
typedef struct expRefList_ *expRefList;  
struct expRefList_ {T_exp *head; expRefList tail;};  
  
struct stmExp {T_stm s; T_exp e;};  
  
static T_stm reorder(expRefList rlist);  
  
static T_stm do_stm(T_stm stm);  
static struct stmExp do_exp(T_exp exp);
```

- **expRefList**: a list of pointers each of which points to a **T_exp**
 - recall that **T_exp** is a pointer to an expression
- **stmExp**: a **T_stm** and a **T_exp**

General Rewriting Rules

```
typedef struct expRefList_ *expRefList;  
struct expRefList_ {T_exp *head; expRefList tail;};  
  
struct stmExp {T_stm s; T_exp e;};  
  
static T_stm reorder(expRefList rlist);  
  
static T_stm do_stm(T_stm stm);  
static struct stmExp do_exp(T_exp exp);
```

- A general procedure during rewriting is: **extracting** the “statement” in a list of expressions, and **updating** the expressions to their ESEQ-clean versions.
- **reorder**: takes a list of expressions, constructs a pair of (**statement**, **expression-list**), return the **statement**, and update the input expressions with the **expression-list**
 - **statement**: the “statement” extracted from the input expressions, which contains all the things that must be executed before the expression-list
 - **expression-list**: the ESEQ-clean version of the input expressions

General Rewriting Rules

```
typedef struct expRefList_ *expRefList;  
struct expRefList_ {T_exp *head; expRefList tail;};  
  
struct stmExp {T_stm s; T_exp e;};  
  
static T_stm reorder(expRefList rlist);  
  
static T_stm do_stm(T_stm stm);  
static struct stmExp do_exp(T_exp exp);
```

- `do_stm`: rewriting a statement and eliminating the ESEQs in this statement.

do_stm

```
/* processes stm so that it contains no ESEQ nodes */
static T_stm do_stm(T_stm stm){
    switch (stm->kind) {
        case T_SEQ: /* SEQ(s1, s2) */
            return seq(do_stm(stm->u.SEQ.left),
                       do_stm(stm->u.SEQ.right));
        case T_JUMP: /* JUMP(e) */
            return seq(reorder(ExpRefList(&stm->u.JUMP.exp, NULL)),
                       stm);
        case T_CJUMP: /* CJUMP(o, e1, e2, t, f) */
            return seq(reorder(ExpRefList(&stm->u.CJUMP.left,
                                           ExpRefList(&stm->u.CJUMP.right, NULL))),
                       stm);
        case T_MOVE: ...
        case T_EXP:
            if (stm->u.EXP->kind == T_CALL)
                return seq(reorder(get_call_rlist(stm->u.EXP)), stm);
            else
                return seq(reorder(ExpRefList(&stm->u.EXP, NULL)), stm);
        default: return stm;
    }
}
```

do_stm

```
/* processes stm so that it contains no ESEQ nodes */
static T_stm do_stm(T_stm stm){
    switch (stm->kind) {
        case T_SEQ: /* SEQ(s1, s2) */
            return seq(do_stm(stm->u.SEQ.left),
                       do_stm(stm->u.SEQ.right));
        case T_JUMP: /* JUMP(e) */
            return seq(reorder(ExpRefList(&stm->u.JUMP.exp, NULL)),
                       stm);
        case T_CJUMP: /* CJUMP(o, e1, e2, t, f) */
            return seq(reorder(ExpRefList(&stm->u.CJUMP.left,
                                           ExpRefList(&stm->u.CJUMP.right, NULL))),
                       stm);
        case T_MOVE: ...
        case T_EXP:
            if (stm->u.EXP->kind == T_CALL)
                return seq(reorder(get_call_rlist(stm->u.EXP)), stm);
            else
                return seq(reorder(ExpRefList(&stm->u.EXP, NULL)), stm);
        default: return stm;
    }
}
```

do_stm (continued)

```
static T_stm do_stm(T_stm stm){
switch (stm->kind) {
...
case T_MOVE:
    if (stm->u.MOVE.dst->kind == T_TEMP &&
        stm->u.MOVE.src->kind == T_CALL)
        return seq(reorder(get_call_rlist(stm->u.MOVE.src)), stm);
    else if (stm->u.MOVE.dst->kind == T_TEMP)
        return seq(reorder(ExpRefList(&stm->u.MOVE.src, NULL)),
                    stm);
    else if (stm->u.MOVE.dst->kind == T_MEM)
        return seq(reorder(ExpRefList(&stm->u.MOVE.dst->u.MEM,
                                       ExpRefList(&stm->u.MOVE.src, NULL))),
                    stm);
    else if (stm->u.MOVE.dst->kind == T_ESEQ) {
        T_stm s = stm->u.MOVE.dst->u.ESEQ.stm;
        stm->u.MOVE.dst = stm->u.MOVE.dst->u.ESEQ.exp;
        return do_stm(T_Seq(s, stm));
    }
    assert(0);
...}}
```

e.g.,

MOVE(ESEQ(sx, e1), e2) =>
SEQ(sx, MOVE(e1, e2))

General Rewriting Rules

```
typedef struct expRefList_ *expRefList;  
struct expRefList_ {T_exp *head; expRefList tail;};  
  
struct stmExp {T_stm s; T_exp e;};  
  
static T_stm reorder(expRefList rlist);  
  
static T_stm do_stm(T_stm stm);  
static struct stmExp do_exp(T_exp exp);
```

- `do_exp`: used by `reorder`
 - `do_exp(e)` returns a statement `s` and an expression `e'`
 - $e \rightarrow \text{ESEQ}(s, e')$
 - `e'` contains no `ESEQs` and `ESEQ(s, e')` would be equivalent to `e`

do_exp

```
static struct stmExp do_exp(T_exp exp){
    switch(exp->kind) {
        case T_BINOP:
            return StmExp(reorder(ExpRefList(&exp->u.BINOP.left,
                                             ExpRefList(&exp->u.BINOP.right, NULL))),
                          exp);
        case T_MEM:
            return StmExp(reorder(ExpRefList(&exp->u.MEM, NULL)),
                          exp);
        case T_ESEQ: {
            struct stmExp x = do_exp(exp->u.ESEQ.exp);
            return StmExp(seq(do_stm(exp->u.ESEQ.stm), x.s), x.e);
        }
        case T_CALL:
            return StmExp(reorder(get_call_rlist(exp)), exp);
        default:
            return StmExp(reorder(NULL), exp);
    }
}
```

do_exp

```
static struct stmExp do_exp(T_exp exp){
    switch(exp->kind) {
        case T_BINOP:
            return StmExp(reorder(ExpRefList(&exp->u.BINOP.left,
                                             ExpRefList(&exp->u.BINOP.right, NULL))),
                          exp);
        case T_MEM:
            return StmExp(reorder(ExpRefList(&exp->u.MEM, NULL)),
                          exp);
        case T_ESEQ: {
            struct stmExp x = do_exp(exp->u.ESEQ.exp);    {x.s, x.e}
            return StmExp(seq(do_stm(exp->u.ESEQ.stm), x.s), x.e);
        }
        case T_CALL:
            return StmExp(reorder(get_call_rlist(exp)), exp);
        default:
            return StmExp(reorder(NULL), exp);
    }
}
```